

Supplementary Material for “Efficient Filtered Vector Search with Diverse Labels: A Trie-Free and Adaptive Routing Approach”

I. AN ALTERNATIVE TRIE-FREE ELS RETRIEVAL METHOD

Algorithm 1 in the main contents first obtains all candidate supersets by intersecting the label-to-group bitmaps. It then examines these candidates in non-decreasing order of label-set cardinality and traverses the LNG descendants of each selected ELS to obtain the exact ELS set $\hat{F}_q$. We present an alternative exact method that keeps the same candidate-generation stage but identifies the ELSs through a complete one-hop scan of the candidate-induced LNG. The two methods have comparable time and space complexities and exhibit similar empirical performance, both substantially outperform the trie-based method. The details are as follows.

Specifics of alternative trie-free ELS retrieval. Similar to Algorithm 1 in the main contents, Algorithm S1 takes as input a non-empty query label set L_q , a collection of label-to-group bitmaps $\mathbb{B}$, and the LNG topology represented by the adjacency lists Child . Each distinct label set $f \in F$ corresponds to a vector group and is assigned a stable integer ID. For each label l , bitmap $\mathbb{B}(l)$ records the groups, *i.e.*, distinct label sets, containing l :

$$\mathbb{B}(l)[f] = 1 \iff l \in f, \quad (\text{S1})$$

where f also denotes its group ID when indexing a bitmap. On the other hand, $\text{Child}[f]$ contains the immediate successors of f on the LNG, representing its minimal strict supersets. The output $\hat{F}_q$ is the exact ELS set.

As in Algorithm 1 of the main contents, Algorithm S1 first selects the query label l^* whose bitmap contains the fewest groups and initializes a candidate bitmap $\mathbb{B}_q^{(a)}$ with a copy of $\mathbb{B}(l^*)$ (Lines 1–2). It then intersects this bitmap with those of the remaining query labels, *i.e.*,

$$\mathbb{B}_q^{(a)} = \bigcap_{l \in L_q} \mathbb{B}(l). \quad (\text{S2})$$

If an intersection becomes empty, the algorithm terminates because no label set satisfies the query. Otherwise, we have

$$\mathbb{B}_q^{(a)}[f] = 1 \iff L_q \subseteq f. \quad (\text{S3})$$

The set bits of $\mathbb{B}_q^{(a)}$ therefore identify exactly the qualified label sets, denoted by $\hat{F}_q^{(a)}$ (Line 7).

Subsequently, the algorithm performs LNG-based descendant pruning and returns the exact ELS set $\hat{F}_q$ (Lines 7–13). It first initializes every candidate in $\hat{F}_q^{(a)}$ as unmarked (Line 8). It then scans each candidate $u \in \hat{F}_q^{(a)}$ and marks every $v \in \text{Child}[u]$ as non-minimal (Lines 9–11). Since an LNG edge (u, v) indicates

Algorithm S1 An alternative trie-free ELS retrieval method

Require: query label set L_q ; label-to-group bitmaps $\mathbb{B}$; LNG topology Child

Ensure: the set $\hat{F}_q$ of entry label sets

```

1:  $l^* \leftarrow \arg \min_{l \in L_q} |\{f \in F \mid \mathbb{B}(l)[f] = 1\}|$ 
2:  $\mathbb{B}_q^{(a)} \leftarrow \mathbb{B}(l^*)$ 
3: for all  $l \in L_q \setminus \{l^*\}$  do
4:    $\mathbb{B}_q^{(a)} \leftarrow \mathbb{B}_q^{(a)} \& \mathbb{B}(l)$ 
5:   if  $\mathbb{B}_q^{(a)} = \emptyset$  then
6:     return  $\emptyset$ 
7:  $\hat{F}_q^{(a)} \leftarrow \{f \mid \mathbb{B}_q^{(a)}[f] = 1\}$ 
8: marked[ $f$ ]  $\leftarrow$  false for every  $f \in \hat{F}_q^{(a)}$ 
9: for all  $u \in \hat{F}_q^{(a)}$  do
10:   for all  $v \in \text{Child}[u]$  do
11:     marked[ $v$ ]  $\leftarrow$  true
12:  $\hat{F}_q \leftarrow \{f \in \hat{F}_q^{(a)} \mid \text{marked}[f] = \text{false}\}$ 
13: return  $\hat{F}_q$ 

```

$u \subsetneq v$, each marked child v has a qualified proper subset u and thus cannot be an ELS. After all candidates have been processed, the algorithm returns the unmarked candidates, which constitute the exact ELS set $\hat{F}_q$ (Lines 12–13).

Correctness of alternative trie-free ELS retrieval. Theorem S1 shows the correctness of Algorithm S1.

Theorem S1. *Algorithm S1 returns the exact ELS set $\hat{F}_q$ for the query label set L_q , *i.e.*,*

$$\hat{F}_q = \{f \in F \mid L_q \subseteq f, \nexists g \in F : L_q \subseteq g \subsetneq f\}. \quad (\text{S4})$$

Proof. Recall that the ELS candidate-generation process of Algorithm S1 is identical to that of Algorithm 1 in the main contents and produces the exact set $\hat{F}_q^{(a)}$ of qualified label sets. Therefore, every candidate retained in $\hat{F}_q$ satisfies the label-containment constraint, *i.e.*, it is a superset of L_q .

We next prove that every candidate retained in F_q is an ELS, *i.e.*, an inclusion-minimal superset of L_q that has no proper subset in F that also contains L_q . We initially prove that the marking stage removes exactly the non-minimal candidates by contradiction. Suppose that a marked candidate $v \in \hat{F}_q^{(a)}$ is an ELS. Candidate v is marked only if there exists a candidate $u \in \hat{F}_q^{(a)}$ such that $v \in \text{Child}[u]$. By the definition of an LNG edge, $u \subsetneq v$. Since u is qualified, $L_q \subseteq u \subsetneq v$, contradicting the assumption that v is an ELS.

We further prove that no non-minimal candidate remains unmarked and is incorrectly returned as an ELS. For any non-minimal candidate v , let u be a maximal qualified proper subset of v . By the construction of the LNG, $v \in \text{Child}[u]$. Because the

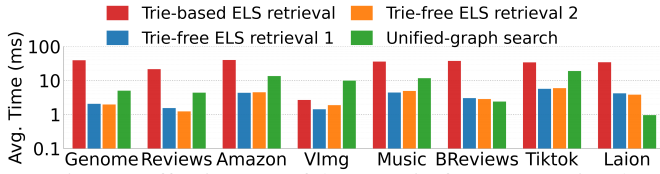


Fig. S1: Effectiveness of the two trie-free ELS retrieval methods.

algorithm processes every candidate, it processes u and marks v . Hence, a candidate remains unmarked if and only if it is an ELS.

Moreover, Algorithm S1 returns every ELS because $\hat{F}_q^{(a)}$ contains all qualified label sets and the algorithm adds every unmarked candidate to $\hat{F}_q$. Therefore, Algorithm S1 returns the exact ELS set. $\square$

Complexity of the alternative trie-free ELS retrieval method.

Algorithm S1 has the following worst-case time complexity, comparable to that of Algorithm 1 in the main contents:

$$O(|F| \cdot |L_q| + |\hat{F}_q^{(a)}| + E),$$

where E denotes the number of LNG edges scanned during the query. The details are as follows. Candidate generation intersects the bitmaps and has a conservative time bound of $O(|F| \cdot |L_q|)$. The subsequent LNG traversal takes $O(|\hat{F}_q^{(a)}| + E)$ time.

As discussed in the main contents, both trie-free ELS retrieval methods achieve lower retrieval latency than the trie-based ELS retrieval method because they replace pairwise containment checks with LNG traversal, whose cost is linear in the number of qualified nodes and scanned edges. As shown in Figure S1, the two trie-free methods achieve comparable retrieval latency and are substantially faster than the trie-based ELS retrieval method. Here, *Trie-free ELS retrieval 1* denotes Algorithm 1 in the main paper, whereas *Trie-free ELS retrieval 2* denotes Algorithm S1 in this section.

Regarding space complexity, Algorithm S1 only changes the LNG descendant-pruning strategy and therefore has the same worst-case space complexity as Algorithm 1 in the main contents. Specifically, both trie-free methods replace the auxiliary trie used by trie-based ELS retrieval with label-to-group bitmaps. The trie requires $O(|\mathcal{L}| \cdot |F|)$ space, whereas the uncompressed bitmaps require $O(|\mathcal{L}| \cdot |F|)$ bits. Thus, the two index structures have the same asymptotic worst-case space bound. We further employ Roaring compression to keep the bitmap storage manageable. The experiments in the main contents demonstrate that this bitmap-based trie-free design scales well to large graphs with diverse labels.

II. ROUTER TRAINING

The main contents show that different LANNS queries favor different search paradigms. Based on this observation, ALPS uses an XGBoost-based router to select among three search paradigms: unified-graph search, proximity-graph search, and pre-filtering brute-force search. The router uses three query-level features, i.e., p_{pass} , $|F_{pass}|$, and $|L_q|$, which capture both vector selectivity and label-set structure. Table SI shows that all three features receive non-negligible importance scores on real

TABLE SI: Feature importance scores in experiments.

Feature	p_{pass}	$ F_{pass} $	$ L_q $
Importance score	0.7088	0.1889	0.1023

datasets, indicating that each feature contributes to the routing decisions. We implement the router with XGBoost because it can capture nonlinear feature interactions while maintaining low inference latency. In our implementation, the maximum tree depth is set to 6, the learning rate is set to 0.05, and the number of boosting rounds is set to 300. All router hyperparameters were fixed before evaluation, and no test queries were used for hyperparameter tuning or early stopping.

We randomly generate 40,000 LANNS queries from all eight datasets, with 5,000 queries from each dataset. These queries exclude the test queries used in the main evaluation. For each query, we evaluate the three candidate paradigms using the same implementations and parameter settings as in the main experiments. Specifically, the unified-graph-based paradigm uses TFNG with $\delta = 6$, the same construction parameter as UNG; the proximity-graph-based paradigm adopts the FAVOR-style search strategy used by ALPS; and the pre-filtering brute-force paradigm exhaustively computes distances over all qualified vectors, thus always achieving a recall of 1. For each paradigm, we follow the same search-queue-length tuning protocol as in the QPS-recall evaluation, varying the maximum search queue length from 10 to 70,000. We then assign the query to the paradigm with the highest QPS among those whose Recall@10 is at least 0.9. If multiple paradigms have the same QPS after measurement rounding, we break ties by first choosing the one with higher recall, and then by a fixed deterministic order: unified-graph-based search, proximity-graph-based search, and pre-filtering.

We generate 5,000 training queries for each of the eight datasets and train a single routing model on the combined 40,000 queries. To evaluate the routing accuracy reported in the main text, we use a separate set of 10,000 test queries for each dataset. These test queries are disjoint from the training queries. We also conduct a leave-one-dataset-out evaluation, where all queries from one dataset are removed from training and the trained model is tested on this held-out dataset. As shown in Tables IV and V in the main contents, the routing accuracy remains above 90% under both settings, indicating that the learned routing rule generalizes across datasets. This accuracy is acceptable, given that the resulting ALPS outperforms state-of-the-art methods in the main experiments.

As shown in Table IV in the main contents, the training time cost is only a few seconds. In all experiments, query latency is measured from receiving the query to returning top- K results, including feature extraction and model prediction. The feature-extraction and prediction overhead is only a few microseconds, showing that the router is suitable for real-time query processing.

III. ADDITIONAL EXPERIMENTS

Figure S2 reports the QPS ratios of our methods to the state-of-the-art baselines at recall ≥ 0.95 . It covers experiments of varying $|L_q|$, p_{pass} , th , and K , where each dot represents the result on one dataset. For a fair comparison, we compute each ratio by dividing the peak QPS of a proposed method by that of

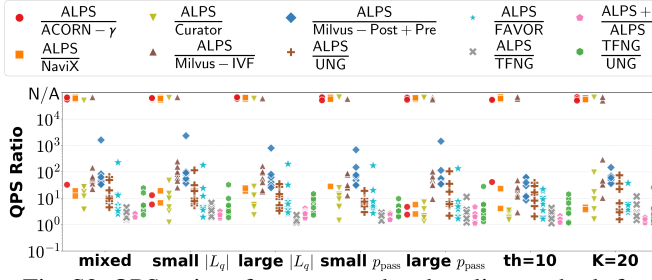


Fig. S2: QPS ratios of our approach to baseline methods for achieving recall ≥ 0.95 .

TABLE SII: Cache hit rates comparison (AP signifies ALPS).

Dataset	Alg.	L1 (%)	L2 (%)	L3 (%)	Dataset	Alg.	L1 (%)	L2 (%)	L3 (%)
Genome	AP	89.95	61.49	81.31	Music	AP	88.51	48.82	35.55
	AP+	89.99	83.80	91.66		AP+	90.53	58.61	73.98
Reviews	AP	84.16	44.68	39.38	BReviews	AP	82.17	60.07	31.41
	AP+	87.76	54.35	77.44		AP+	88.67	60.71	74.63
Amazon	AP	88.79	56.14	27.64	Tiktok	AP	72.88	96.95	18.16
	AP+	89.84	61.57	64.11		AP+	73.44	98.04	57.04
VImg	AP	84.32	55.75	18.42	Laion	AP	80.88	59.43	49.95
	AP+	88.15	56.39	62.47		AP+	86.46	65.00	71.59

the corresponding baseline, subject to both methods achieving recall ≥ 0.95 . If a baseline cannot reach this recall under a given setting, we report the corresponding ratio as N/A. As shown in Figure S2, the QPS ratios of ALPS to the baselines are often above 10. These results confirm our observation in the main contents that, for online queries over workloads with diverse labels, ALPS often delivers more than $10\times$ the QPS of state-of-the-art methods at matched high recall.

IV. CACHE HIT RATES COMPARISON

Frequent switching among search paradigms in ALPS may reduce cache locality and make repeated data accesses a major performance bottleneck. Therefore, ALPS+ groups queries with similar execution behavior to improve memory-access locality. Table SII reports the cache hit rates of ALPS and ALPS+, where L1, L2, and L3 denote the three CPU cache levels. ALPS+ achieves higher hit rates at all three levels, indicating that query grouping effectively improves cache locality and contributes to its higher throughput.